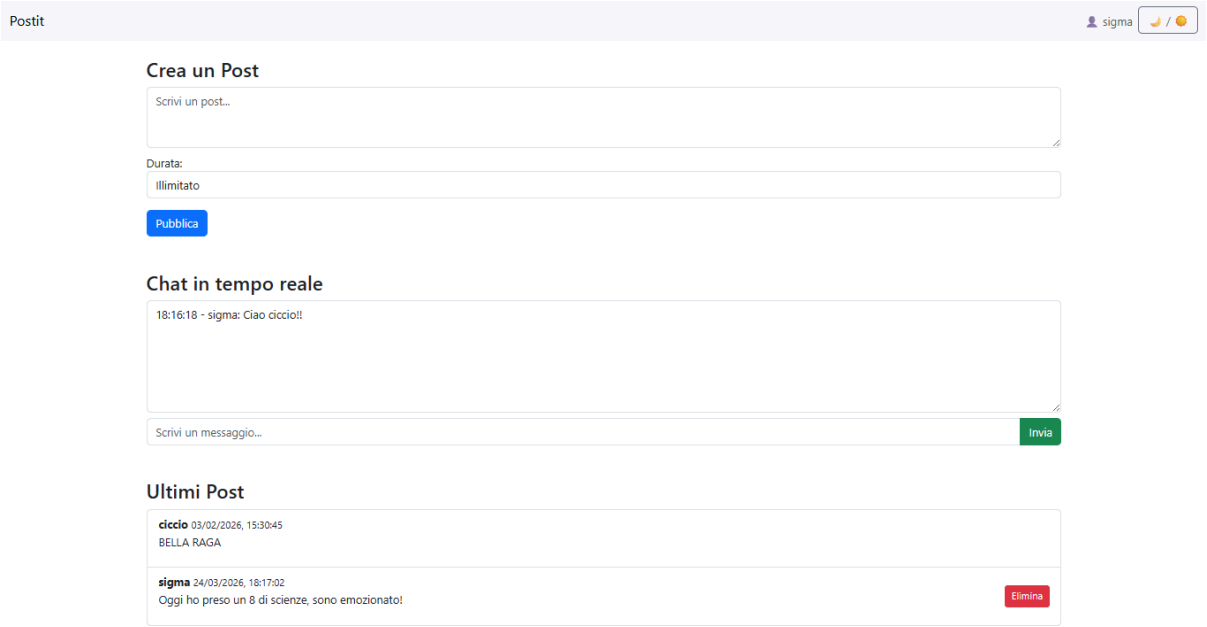
# Documentazione tecnica "Social post-it" v2




## 1. Introduzione

### 1.1 Scopo del Software

Il software "Social Post It" è un'applicazione web che consente agli utenti autenticati di pubblicare post con una durata temporale limitata o illimitata. L'obiettivo principale è creare un ambiente interattivo in cui gli utenti possano condividere messaggi che scadono automaticamente dopo un certo periodo di tempo e comunicare in tempo reale tramite una chat integrata.

### 1.2 Descrizione Generale del Software

Il software è sviluppato utilizzando Node.js con il framework Express e utilizza EJS per il rendering delle pagine. I dati degli utenti e dei post sono memorizzati in file JSON. La versione 2 introduce importanti novità rispetto alla precedente:

- Autenticazione basata su cookie: gli utenti devono effettuare il login per accedere all'applicazione e la sessione viene mantenuta tramite cookie HTTP.
- Middleware di protezione delle route: tutte le pagine principali sono protette da un middleware requireLogin che reindirizza automaticamente al login gli utenti non autenticati.
- Eliminazione dei propri post: ogni utente loggato può eliminare i propri post tramite un apposito pulsante.
- Chat in tempo reale tramite WebSocket: gli utenti possono comunicare istantaneamente tra loro grazie all'integrazione della libreria ws.

- Identificazione univoca dei post tramite ID timestamp.

### 1.3 Obiettivi della Documentazione

Questa documentazione descrive l'architettura, il design e il funzionamento del software v2, fornendo dettagli tecnici e spiegazioni del codice aggiornato rispetto alla versione precedente.

### 1.4 Panoramica del Documento

Il documento copre i seguenti aspetti:

- Architettura del software
- Analisi delle proprietà (robustezza, usabilità, portabilità)
- Design del software
- Documentazione del codice
- Testing
- Deployment e manutenzione

## 2. Architettura del Software

### 2.1 Descrizione Generale dell'Architettura

L'architettura del software segue un modello MVC semplificato, arricchito con un layer di comunicazione real-time:

- Model: I dati vengono gestiti tramite file JSON (users.json, posts.json). La chat è gestita in memoria (array chat).
- View: Le interfacce utente sono realizzate con EJS e Bootstrap.
- Controller: Le route Express gestiscono le richieste HTTP; il server WebSocket gestisce la comunicazione bidirezionale in tempo reale.

### 2.2 Componenti Principali

- Express.js per la gestione delle richieste HTTP.
- EJS per la generazione dinamica delle pagine HTML.
- File JSON per la memorizzazione di utenti e post.
- bcryptjs per la crittografia delle password.
- cookie-parser per la gestione dei cookie di sessione.
- ws (WebSocket) per la chat in tempo reale tra utenti.

### 2.3 Flusso di Dati

1. L'utente si registra o effettua il login; al login avvenuto con successo, viene impostato un cookie con lo username.
2. Il middleware requireLogin verifica la presenza del cookie prima di consentire l'accesso a qualsiasi pagina protetta.
3. L'utente autenticato può pubblicare un post con durata definita oppure illimitata.
4. I post scaduti vengono rimossi automaticamente ogni 5 secondi.
5. L'utente può eliminare i propri post tramite un apposito bottone.
6. La chat in tempo reale è attiva tramite WebSocket: i messaggi vengono distribuiti a tutti i client connessi istantaneamente.

## 3. Analisi delle Proprietà

### 3.1 Robustezza

- Gestione degli errori: il codice verifica l'esistenza di utenti duplicati in fase di registrazione e reindirizza all'apposita pagina in caso di credenziali errate.
- Protezione delle route: il middleware requireLogin impedisce l'accesso alle pagine principali senza autenticazione.
- Resilienza: i post vengono caricati da file JSON e gestiti in memoria per efficienza.
- Codice documentato per la gestione degli errori: la verifica della password utilizza bcrypt.compareSync().
- Identificazione univoca dei post: ogni post riceve un ID univoco basato su Date.now(), che permette l'eliminazione sicura senza ambiguità.

### 3.2 Usabilità

- Interfaccia utente chiara: utilizzo di Bootstrap per un design responsive.
- Accessibilità: supporto al tema scuro/chiaro con localStorage.
- Feedback agli utenti: lo username loggato è sempre visibile nella navbar.
- Chat integrata: gli utenti comunicano in tempo reale senza ricaricare la pagina.
- Gestione dei propri contenuti: il pulsante 'Elimina' appare solo sui post dell'utente corrente.

### 3.3 Portabilità

- Compatibilità: funziona su qualsiasi server Node.js.
- Gestione della configurazione: il codice può essere facilmente modificato per supportare database.

## 4. Design del Software

### 4.1 Principi di Design Adottati

- Separazione tra logica di business e presentazione.
- Uso di funzioni modulari per caricamento/salvataggio dati.
- Middleware riutilizzabile (requireLogin) per la protezione delle route.

### 4.2 Scelte Architetturali e Motivi

- Scelta di JSON per la memorizzazione: semplicità e rapidità di sviluppo.
- Cookie HTTP per la sessione: soluzione leggera e senza dipendenze aggiuntive rispetto all'uso di express-session.
- WebSocket (ws) per la chat: comunicazione bidirezionale a bassa latenza, ideale per applicazioni real-time.
- Chat in memoria (array): scelta semplice per un prototipo; non persiste al riavvio del server.

## 5. Documentazione del Codice

### 5.1 Struttura del Codice Sorgente

- app.js — Entry point del server.
- views/ — Contiene i file EJS (index.ejs, login.ejs, register.ejs).
- public/ — Contiene CSS, immagini e le due librerie di Bootstrap (ora in sottocartella /bootstrap/).
- users.json — Archivio utenti.
- posts.json — Archivio post.

### 5.2 Classi e Metodi Principali

- loadUsers() — Carica gli utenti da users.json.
- loadPosts() — Carica i post da posts.json.
- savePosts(posts) — Salva i post su posts.json.
- cleanExpiredPosts() — Rimuove i post scaduti.
- requireLogin(req, res, next) — Middleware che protegge le route autenticate.

## 5.3 Struttura di ogni codice e spiegazione riga per riga

### 5.3.1 app.js

**Importazioni e configurazione iniziale:**

```
// Importazione moduli
const express = require('express');
const fs = require('fs');
const path = require('path');
const bcrypt = require('bcryptjs');
const bodyParser = require('body-parser');
const cookieParser = require('cookie-parser');
const { Server } = require('ws');
```

Rispetto alla versione precedente, sono state aggiunte due nuove dipendenze:

- **cookie-parser:** middleware per leggere e scrivere i cookie HTTP nella richiesta/risposta. Necessario per la gestione della sessione utente.
- **ws (WebSocket):** libreria per la creazione del server WebSocket. Viene importata tramite la destrutturazione { Server }, che estrae la classe Server dal modulo ws.

**Middleware applicati:**

```
app.use(bodyParser.urlencoded({ extended: false }));
app.use(cookieParser());
app.use(express.static(path.join(__dirname, 'public')));
app.set('view engine', 'ejs');
```

Rispetto alla versione precedente, viene aggiunto:

- **app.use(cookieParser()):** abilita il parsing automatico dei cookie presenti nell'header HTTP di ogni richiesta, rendendo disponibili i dati tramite req.cookies.

**Funzioni di accesso ai dati:**

```
// ---------------- USERS ----------------

function loadUsers() {
    return JSON.parse(fs.readFileSync(path.join(__dirname, 'users.json')));
}

// ---------------- POSTS ----------------

function loadPosts() {
    return JSON.parse(fs.readFileSync(path.join(__dirname, 'posts.json')));
}

function savePosts(posts) {
    fs.writeFileSync(path.join(__dirname, 'posts.json'), JSON.stringify(posts, null, 2));
}

function cleanExpiredPosts() {
    let posts = loadPosts();
    const now = Date.now();
    posts = posts.filter(post => post.expiryTime === null || post.expiryTime > now);
    savePosts(posts);
}

setInterval(cleanExpiredPosts, 5000);
```

Queste funzioni rimangono invariate rispetto alla versione precedente. cleanExpiredPosts() viene chiamata automaticamente ogni 5 secondi tramite setInterval.

**Middleware requireLogin:**

```
// ---------------- LOGIN PROTECTION ----------------

function requireLogin(req, res, next) {
    if (!req.cookies.username) {
        return res.redirect('/login');
    }
    next();
}
```

Questa è una delle principali novità della v2. Il middleware requireLogin:

- Controlla se il cookie username è presente nella richiesta tramite req.cookies.username.
- Se il cookie non esiste (utente non loggato), reindirizza immediatamente alla pagina /login.
- Se il cookie è presente, chiama next() per consentire l'elaborazione normale della route.

Questo middleware viene applicato direttamente sulle route che richiedono autenticazione, passandolo come secondo argomento prima della callback della route.

**Route GET /:**

```
// ---------------- ROUTES ----------------

app.get('/', requireLogin, (req, res) => {
    cleanExpiredPosts();
    const posts = loadPosts();
    res.render('index', { posts, username: req.cookies.username });
});

app.get('/login', (req, res) => res.render('login'));
app.get('/register', (req, res) => res.render('register'));
```

La home page ora richiede il login obbligatorio (requireLogin come secondo parametro). Viene caricata la lista dei post dopo aver rimosso quelli scaduti, e viene passato alla view anche lo username recuperato dal cookie (req.cookies.username), in modo che la pagina possa mostrarlo nella navbar e usarlo per le operazioni sui post.

**Route POST /login — gestione cookie:**

```
app.post('/login', (req, res) => {
    const { username, password } = req.body;
    const users = loadUsers();
    const user = users.find(u => u.username === username);

    if (user && bcrypt.compareSync(password, user.password)) {
        res.cookie("username", username);
        res.redirect('/');
    } else {
        res.redirect('/login');
    }
});
```

Rispetto alla versione precedente, in caso di credenziali corrette non ci si limita a fare redirect alla home, ma viene prima impostato il cookie username con il valore dello username dell'utente tramite res.cookie("username", username). Questo cookie sarà letto da tutte le route successive per identificare l'utente loggato.

**Route POST /post — pubblicazione con ID:**

```
app.post('/post', requireLogin, (req, res) => {
    const username = req.cookies.username;
    const { content, timer } = req.body;

    const expiryTime = timer === "unlimited" ? null : Date.now() + parseInt(timer);

    const posts = loadPosts();
    posts.push({
        id: Date.now(),   // ID UNICO
        username,
        content,
        date: new Date().toISOString(),
        expiryTime
    });

    savePosts(posts);
    res.redirect('/');
});
```

La route di pubblicazione post è ora protetta da requireLogin. Lo username non viene più preso dal corpo del form ma direttamente dal cookie (req.cookies.username), evitando così che un utente possa pubblicare a nome di altri. Inoltre, ogni post riceve ora un campo id univoco calcolato tramite Date.now(), che rappresenta il timestamp in millisecondi. Questo ID è fondamentale per la funzionalità di eliminazione.

**Route DELETE — eliminazione post:**

```
// ---------------- DELETE POST ----------------

app.get('/delete-post/:id', requireLogin, (req, res) => {
    const id = parseInt(req.params.id);
    let posts = loadPosts();

    posts = posts.filter(p => p.id !== id);

    savePosts(posts);
    res.redirect('/');
});
```

Questa è una funzionalità completamente nuova nella v2. La route accetta come parametro dinamico :id (l'ID del post da eliminare). Il flusso è:

7. Viene estratto il parametro id dall'URL tramite req.params.id e convertito in numero intero con parseInt().
8. Viene caricata la lista di tutti i post.
9. Viene filtrato l'array escludendo il post il cui id corrisponde a quello richiesto (posts.filter(p => p.id !== id)).
10. La lista aggiornata viene salvata su posts.json.
11. L'utente viene reindirizzato alla home.

Nota: la route non verifica se il richiedente è il proprietario del post lato server; il controllo visuale (mostrare il bottone solo al proprietario) è effettuato nel template EJS.

**Server HTTP e WebSocket:**

```
// ---------------- SERVER + WEBSOCKET ----------------

const server = app.listen(PORT, () =>
    console.log(`Server attivo su http://localhost:${PORT}`)
);

const wsServer = new Server({ server });

let chat = [];

wsServer.on('connection', ws => {
    ws.send(JSON.stringify(chat));

    ws.on('message', msg => {
        const data = JSON.parse(msg);

        const riga = {
            username: data.username,
            text: data.text,
            time: new Date().toLocaleTimeString()
        };

        chat.push(riga);

        wsServer.clients.forEach(client => {
            if (client.readyState === 1) {
                client.send(JSON.stringify(chat));
            }
        });
    });
});
```

Nella v2 il server viene avviato in modo diverso rispetto alla versione precedente, per permettere la condivisione della stessa porta tra HTTP e WebSocket:

- Il server HTTP viene salvato in una variabile server (const server = app.listen(...)) invece di essere avviato e dimenticato.
- Il server WebSocket viene creato passando il server HTTP come opzione: new Server({ server }). Questo fa sì che le connessioni WebSocket e HTTP condividano la porta 3000.
- L'array chat è una variabile globale in memoria che contiene tutti i messaggi della sessione corrente.
- Quando un client si connette (evento connection), il server invia immediatamente l'intero storico della chat con ws.send(JSON.stringify(chat)).
- Quando arriva un messaggio (evento message), viene costruito un oggetto con username, text e time (ora corrente formattata), aggiunto all'array chat e trasmesso a tutti i client connessi con readyState === 1 (connessione aperta).

### 5.3.2 index.ejs

**Struttura Generale:**

Il file index.ejs è la pagina principale dell'applicazione. Nella v2 è profondamente rinnovato rispetto alla versione precedente: non contiene più i link di login/registrazione, include la sezione di chat WebSocket, e mostra il bottone di eliminazione sui propri post.

```
// WEBSOCKET CHAT
const ws = new WebSocket(location.origin.replace(/^http/, 'ws'));

ws.onmessage = event => {
    const data = JSON.parse(event.data);
    let testo = "";

    data.forEach(msg => {
        testo += msg.time + " - " + msg.username + ": " + msg.text + "\n";
    });

    const area = document.getElementById("chatArea");
    area.value = testo;
    area.scrollTop = area.scrollHeight;
};

function inviaChat() {
    const text = document.getElementById("chatText").value.trim();
    if (!text) return;

    ws.send(JSON.stringify({
        username: "<%= username %>",
        text: text
    }));

    document.getElementById("chatText").value = "";
}
```

**Navbar:**

La navbar non mostra più i link Login e Registrati. Al loro posto compare lo username dell'utente loggato (ricevuto dal server come variabile EJS), preceduto dall'icona 👤. Questo rende immediatamente visibile all'utente con quale account sta operando. Il pulsante per il cambio tema è sempre presente.

**Form di pubblicazione post:**

Il form di pubblicazione è simile alla versione precedente. Non è più presente il campo nascosto per lo username (che nella v1 veniva inviato dal form), perché nella v2 lo username viene letto direttamente dal cookie lato server.

**Sezione Chat in tempo reale:**

Questa sezione è completamente nuova. È composta da:

- Una textarea con id chatArea in modalità sola lettura (readonly), che visualizza i messaggi della chat in formato testuale.
- Un campo di input con id chatText per scrivere il messaggio.
- Un pulsante 'Invia' che chiama la funzione JavaScript inviaChat() al click.

**Lista dei post con pulsante Elimina:**

La lista dei post è arricchita con un controllo condizionale EJS:

- Ogni post viene visualizzato con username, data e contenuto come prima.
- Un blocco condizionale <% if (post.username === username) { %> controlla se il post appartiene all'utente loggato. In caso affermativo, viene mostrato un link con classe btn btn-danger che punta alla route /delete-post/<post.id>. In caso contrario, il pulsante non viene renderizzato.

**Script WebSocket:**

Il blocco <script> nella v2 gestisce la connessione WebSocket con il server:

- new WebSocket(location.origin.replace(/^http/, 'ws')): crea la connessione WebSocket convertendo automaticamente l'URL della pagina da http:// a ws://.
- ws.onmessage: ogni volta che il server invia dati, viene ricostruito il testo della chat iterando sui messaggi ricevuti (data.forEach) e concatenando ora, username e testo. Il testo viene inserito nella textarea e questa viene scorsa automaticamente all'ultimo messaggio tramite area.scrollTop = area.scrollHeight.
- inviaChat(): legge il testo dal campo di input, verifica che non sia vuoto, e invia al server un oggetto JSON con username (valore iniettato da EJS con il valore del cookie) e text. Dopo l'invio, il campo viene svuotato.

Nota: a differenza delle versioni login.ejs e register.ejs, in questo file non è presente lo script per il toggle del tema scuro/chiaro.

### 5.3.3 login.ejs

**Struttura Generale:**

Il file login.ejs è sostanzialmente simile alla versione precedente. Le differenze principali sono:

- Il percorso del file CSS Bootstrap è aggiornato a /bootstrap/bootstrap.min.css (Bootstrap ora è nella sottocartella bootstrap/ all'interno di public/).
- Il form invia i dati alla route /login con metodo POST.
- Lo script per il tema scuro/chiaro è presente e funzionante (identico alla versione precedente).

### 5.3.4 register.ejs

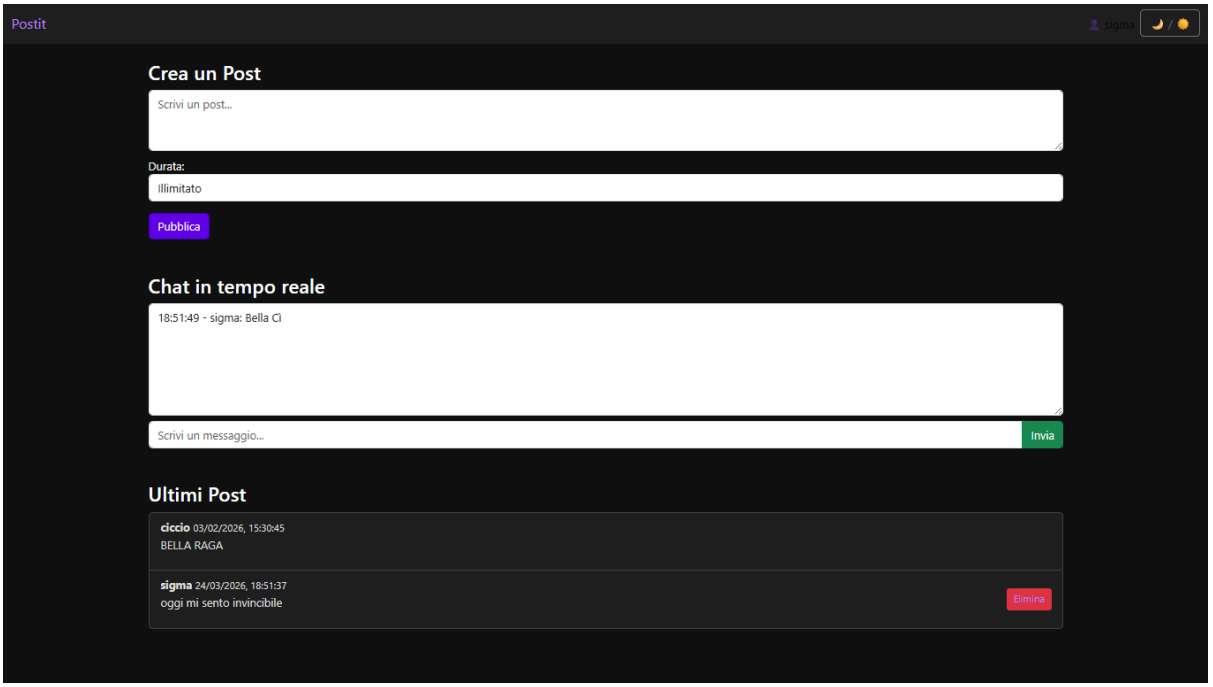
**Struttura Generale:**

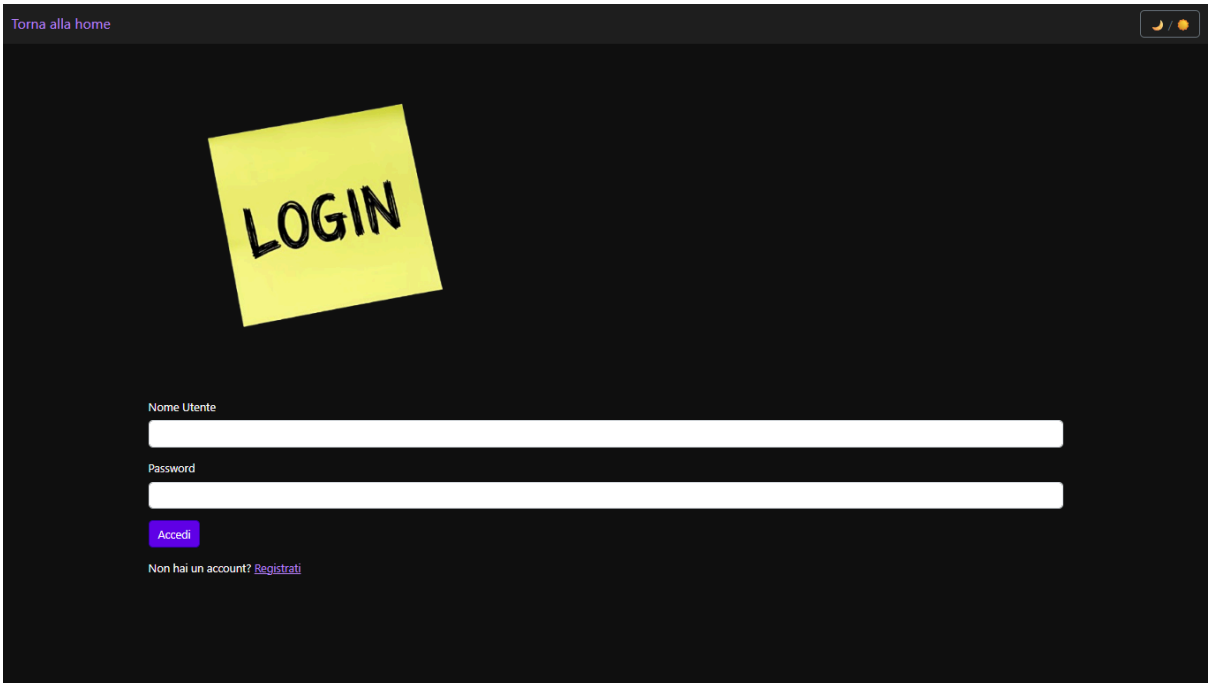
Il file register.ejs è anch'esso molto simile alla versione precedente. Le modifiche sono le stesse di login.ejs: aggiornamento del percorso Bootstrap e script tema scuro/chiaro presente e funzionante.

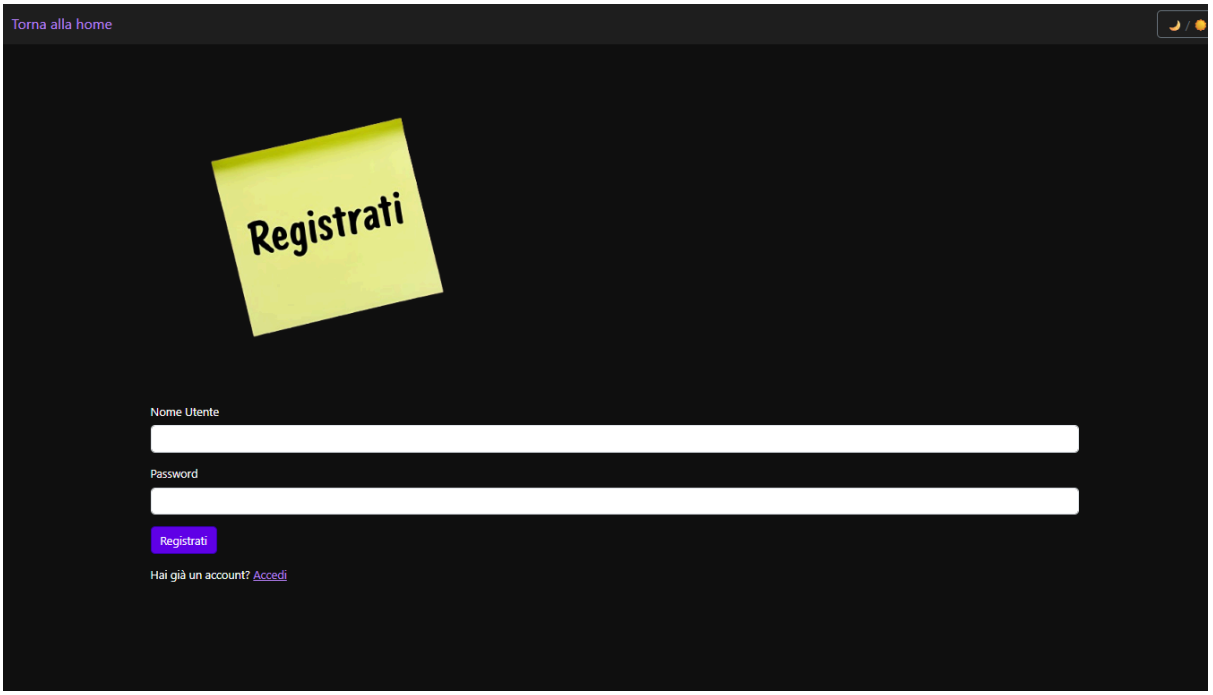
### 5.3.5 darkmode.css

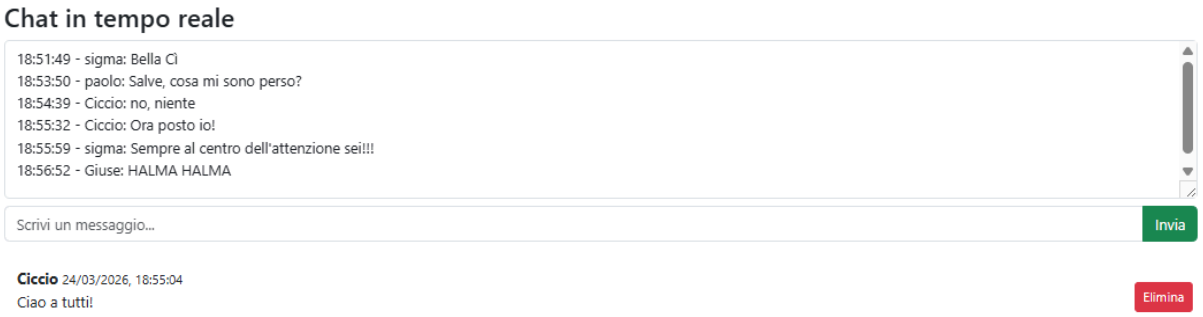
Il file darkmode.css non ha subito modifiche rispetto alla versione precedente. Continua a gestire la classe .dark-mode con sfondo scuro (#121212), navbar grigia (#1f1f1f), elementi lista con sfondo #222, link viola (#bb86fc) e pulsanti primari viola (#6200ea).

# 6. Videate






# 7. Testing

## 7.1 Strategia di Testing

Sono stati effettuati test manuali sulle principali funzionalità, con particolare attenzione alle nuove feature introdotte nella v2.

## 7.2 Test Funzionali Principali

- Registrazione: verifica della creazione corretto dell'account e del blocco in caso di username duplicato.
- Login / Logout: verifica che il cookie venga impostato correttamente al login e che le route protette siano inaccessibili senza cookie.
- Pubblicazione post: verifica che lo username venga letto dal cookie e non dal form.
- Scadenza automatica: verifica della rimozione dei post scaduti dopo il tempo indicato.
- Eliminazione post: verifica che un utente possa eliminare solo i propri post.
- Chat WebSocket: verifica della ricezione in tempo reale dei messaggi su più schede/browser aperti contemporaneamente.

## 7.3 Test di Integrazione

Test manuali effettuati su diverse configurazioni di browser e dispositivi. La chat è stata testata con connessioni multiple simultanee per verificare la broadcast corretta dei messaggi.

### 7.4 Test di Performance e Stress

Verificato il corretto funzionamento con centinaia di post caricati. La chat in memoria non è persistente e si azzera al riavvio del server.

# 8. Deployment e Portabilità

### 8.1 Guida al Deployment

12. Installare Node.js (versione >= 14 raccomandata per il supporto WebSocket nativo).
13. Clonare il repository.
14. Eseguire npm install per installare le dipendenze (express, ejs, bcryptjs, body-parser, cookie-parser, ws).
15. Avviare il server con node app.js.
16. Aprire il browser su http://localhost:3000.

### 8.2 Dipendenze aggiuntive rispetto alla v1

Rispetto alla versione precedente, il package.json include due nuove dipendenze:

- **cookie-parser ^1.4.7:** per la gestione dei cookie di sessione.
- **ws ^8.19.0:** per la comunicazione WebSocket in tempo reale.

# 9. Manutenzione del Software

### 9.1 Strategie di Manutenzione

- Controllo periodico della sicurezza degli utenti.
- Ottimizzazione delle prestazioni per grandi volumi di dati.
- Monitoraggio della crescita dell'array chat in memoria per evitare memory leak in sessioni molto lunghe.

### 9.2 Aggiornamenti del Software

Aggiornamenti previsti per le versioni future:

- Implementare la persistenza della chat (salvataggio su file JSON o database).
- Aggiungere una route /logout che cancella il cookie di sessione.
- Integrare un database SQL o NoSQL per scalabilità.
- Aggiungere il controllo lato server sull'ownership del post prima dell'eliminazione.

### 9.3 Bug Fixing e Refactoring

Monitoraggio dei log per identificare problemi di scalabilità. Aggiungere lo script del tema scuro/chiaro anche in index.ejs (attualmente mancante).

# 10. Conclusioni

### 10.1 Sintesi delle Proprietà Analizzate

Il software v2 mantiene le caratteristiche di robustezza, usabilità e portabilità della versione precedente, aggiungendo autenticazione basata su cookie, comunicazione in tempo reale tramite WebSocket e la gestione dei propri contenuti da parte degli utenti.

## 10.2 Suggerimenti Futuri per il Miglioramento

- Implementare una route /logout per permettere agli utenti di disconnettersi.
- Aggiungere la persistenza della chat su file JSON o database.
- Integrare un database SQL o NoSQL per migliorare la scalabilità.
- Aggiungere validazione e sanitizzazione dell'input per prevenire attacchi XSS.
- Implementare il controllo lato server dell'ownership del post prima dell'eliminazione.

## 10.3 Lezioni Apprese durante lo Sviluppo

L'uso di cookie HTTP per la sessione è una soluzione semplice ed efficace per applicazioni di piccole dimensioni, ma per progetti più grandi è consigliabile adottare express-session con store persistente. L'integrazione di WebSocket con Express su un singolo server condiviso è elegante e non richiede una porta separata.